

Simulador de Línea de Ensamblaje - Fases 3, 4 y 5

Estado Actual del Proyecto

Completado (Fases 1 y 2)

- **Fase 1:** Estructuras de datos y patrones de diseño implementados
 - Factory Pattern para creación de productos
 - Chain of Responsibility preparado para estaciones
 - Strategy Pattern diseñado para algoritmos de scheduling
 - Estructuras de datos base (Product, Queue thread-safe)
- **Fase 2:** Sistema de métricas básico implementado
 - Sistema de logging multi-nivel con colores
 - Recolección de métricas por estación y globales
 - Sistema preparado para concurrencia (macros condicionales)
 - Registro de eventos y estadísticas

Archivos Actuales

proyecto/	
├── core/	
│ ├── product.h/c	(Completado)
│ ├── queue.h/c	(Completado – Thread-safe)
│ ├── station.h/c	(Implementado – Pendiente integración)
│ └── scheduler.h	(Definido – Pendiente implementación)
├── patterns/	
│ ├── factory.h/c	(Completado)
│ ├── chain_handler.h	(Definido – Pendiente implementación)
│ └── strategy.h	(Definido – Pendiente implementación)
├── metrics/	
│ ├── metrics.h/c	(Completado – Sin threading activo)
│ └── logger.h/c	(Completado – Sin threading activo)
└── main.c	(Versión secuencial)

Fase 3: Agregar Concurrencia (Hilos)

Objetivo

Convertir el sistema secuencial en un sistema concurrente donde cada estación opera como un hilo independiente que procesa productos de manera autónoma.

3.1 Implementación de Estaciones como Hilos

Archivos a completar:

- `core/station.c` - Implementación completa del worker thread

Tareas específicas:

3.1.1 Worker Thread de Estación

- Implementar función `station_worker_thread()` que:
 - Ejecuta loop infinito mientras `thread_running` sea verdadero
 - Espera productos de `input_queue` de manera bloqueante (`queue_pop`)
 - Procesa producto con exclusión mutua usando semáforo
 - Envía producto procesado a `output_queue` o siguiente estación
 - Registra métricas automáticamente

3.1.2 Sincronización de Estación

- Usar `sem_wait()` antes de procesar para garantizar un solo producto a la vez
- Proteger acceso a `station->state` con `pthread_mutex_lock/unlock`
- Usar `sem_post()` después de procesar para liberar recurso
- Implementar shutdown limpio con `pthread_cond_signal` si es necesario

3.1.3 Simulación de Procesamiento

- Implementar `station_simulate_processing()`:
 - Calcular tiempo real = `processing_time_ms` + variación aleatoria
 - Usar `usleep(tiempo_ms * 1000)` para simular trabajo
 - Variación aleatoria: $\pm 25\%$ del tiempo base

3.1.4 Integración con Métricas

- Llamar `record_station_entry()` al iniciar procesamiento
- Llamar `record_station_exit()` al finalizar procesamiento
- Actualizar estadísticas internas de la estación
- Registrar eventos en el logger con nivel INFO

3.2 Activación del Sistema Thread-Safe

Archivos a modificar:

- `Makefile` - Agregar flag `-DUSE_THREADING`
- `metrics/metrics.c` - Los locks se activarán automáticamente
- `metrics/logger.c` - Los locks se activarán automáticamente

Tareas específicas:

3.2.1 Modificación del Makefile

```
# Cambiar línea de CFLAGS:
CFLAGS = -Wall -Wextra -pthread -g -DUSE_THREADING
```

3.2.2 Verificación de Sincronización

- Verificar que `METRICS_LOCK()` y `METRICS_UNLOCK()` se ejecuten
- Verificar que `LOGGER_LOCK()` y `LOGGER_UNLOCK()` se ejecuten
- Probar que múltiples hilos puedan escribir métricas sin race conditions

3.3 Chain of Responsibility con Hilos

Archivos a crear:

- `patterns/chain_handler.c`

Tareas específicas:

3.3.1 Implementar Handler Base

- Crear función `create_processing_handler()` genérica
- Implementar `handle()` que llama a la función específica del handler
- Configurar enlaces `next_handler` para formar la cadena

3.3.2 Handlers Específicos

- `create_cutting_handler()` - Para estación de corte
- `create_assembly_handler()` - Para estación de ensamblaje
- `create_packaging_handler()` - Para estación de empaque
- Cada uno con configuración específica (velocidad, complejidad, etc.)

3.3.3 Procesamiento a través de la Cadena

- Implementar `process_through_chain()`:
 - Verifica si handler puede procesar (`can_handle()`)
 - Llama a `handle()` del handler actual
 - Si no puede o falla, pasa a `next_handler`
 - Retorna resultado del procesamiento

3.4 Integración en Main

Archivo a modificar:

- `main.c`

Tareas específicas:

3.4.1 Crear Colas de Comunicación

```
queue_t *queue_cutting = queue_init();
queue_t *queue_assembly = queue_init();
queue_t *queue_packaging = queue_init();
queue_t *queue_output = queue_init();
```

3.4.2 Crear y Configurar Estaciones

```
station_t *cutting = create_station(STATION_TYPE_CUTTING, "Corte", 2000);
station_set_input_queue(cutting, queue_cutting);
station_set_output_queue(cutting, queue_assembly);

station_t *assembly = create_station(STATION_TYPE_ASSEMBLY, "Ensamblaje",
3000);
station_set_input_queue(assembly, queue_assembly);
station_set_output_queue(assembly, queue_packaging);

station_t *packaging = create_station(STATION_TYPE_PACKAGING, "Empaque",
1000);
station_set_input_queue(packaging, queue_packaging);
station_set_output_queue(packaging, queue_output);
```

3.4.3 Iniciar Hilos de Estaciones

```
station_start_thread(cutting);
station_start_thread(assembly);
station_start_thread(packaging);
```

3.4.4 Agregar Productos al Sistema

```
for (int i = 0; i < num_products; i++) {
    product_t *p = factory_create_product(factory);
    queue_push(queue_cutting, p); // Enviar a primera estación
}
```

3.4.5 Esperar Completitud y Cleanup

```
// Esperar a que todas las colas se vacíen
while (queue_output->count < num_products) {
    sleep(1);
}

// Detener hilos
station_stop_thread(cutting);
station_stop_thread(assembly);
station_stop_thread(packaging);

// Destruir recursos
destroy_station(cutting);
destroy_station(assembly);
destroy_station(packaging);
```

```
queue_destroy(queue_cutting);  
// ... etc
```

3.5 Pruebas de Concurrencia

Verificaciones necesarias:

3.5.1 Verificar Thread Safety

- Ejecutar con 10+ productos y verificar que no haya race conditions
- Usar `valgrind --tool=helgrind` para detectar race conditions
- Verificar que métricas sean consistentes

3.5.2 Verificar Deadlock-Free

- Probar con diferentes números de productos (1, 10, 50, 100)
- Verificar que el sistema siempre termine
- Usar timeout en las pruebas

3.5.3 Verificar Logs

- Revisar que logs de diferentes hilos no se entrelacen
- Verificar timestamps correctos
- Confirmar que eventos se registren en orden lógico

Fase 4: Implementar Algoritmos de Scheduling Reales

Objetivo

Implementar un scheduler real que controle el orden de procesamiento de productos usando FCFS y Round Robin con preemption.

4.1 Implementación del Scheduler Base

Archivos a completar:

- `core/scheduler.c`

Tareas específicas:

4.1.1 Crear Scheduler

```
scheduler_t *create_scheduler(scheduling_algorithm_t algorithm, int  
quantum_ms) {  
    // Asignar memoria  
    // Inicializar mutex, condition variables, semáforos  
    // Crear colas (ready_queue, waiting_queue)  
    // Configurar algoritmo
```

```
    // Inicializar estadísticas  
}
```

4.1.2 Worker Thread del Scheduler

```
void *scheduler_worker_thread(void *arg) {  
    while (scheduler->thread_running) {  
        // Esperar productos en ready_queue  
        // Seleccionar siguiente producto según algoritmo  
        // Despachar a primera estación  
        // Actualizar estadísticas  
        // Registrar métricas  
    }  
}
```

4.1.3 Funciones de Control

- `scheduler_add_product()` - Agregar producto a ready_queue
- `scheduler_dispatch_product()` - Enviar producto a primera estación
- `scheduler_context_switch()` - Cambiar de producto activo
- `scheduler_start_thread()` - Iniciar hilo del scheduler
- `scheduler_stop_thread()` - Detener limpiamente

4.2 Implementación de FCFS

Archivo: `core/scheduler.c`

Tareas específicas:

4.2.1 Selección de Producto

```
product_t *scheduler_fcfs_select_next(scheduler_t *scheduler) {  
    // Obtener primer producto de ready_queue (FIFO estricto)  
    return queue_pop(scheduler->ready_queue);  
}
```

4.2.2 Procesamiento FCFS

```
void scheduler_fcfs_process(scheduler_t *scheduler) {  
    while (scheduler->thread_running) {  
        product_t *product = scheduler_fcfs_select_next(scheduler);  
        if (product) {  
            scheduler_dispatch_product(scheduler, product);  
            scheduler->stats.total_products_scheduled++;  
        }  
    }  
}
```

```
}  
}
```

4.2.3 Características de FCFS

- Sin preemption
- Orden de llegada estricto
- Sin quantum
- Simple pero puede causar starvation

4.3 Implementación de Round Robin

Archivo: **core/scheduler.c**

Tareas específicas:

4.3.1 Control de Quantum

```
void scheduler_rr_start_quantum(scheduler_t *scheduler, product_t  
*product) {  
    // Guardar tiempo de inicio  
    clock_gettime(CLOCK_MONOTONIC, &scheduler-  
>quantum_control.quantum_start_time);  
  
    // Configurar quantum  
    scheduler->quantum_control.quantum_remaining_ms = scheduler-  
>config.quantum_ms;  
    scheduler->quantum_control.quantum_expired = 0;  
  
    // Configurar tiempo restante del producto  
    product->remaining_time = scheduler->config.quantum_ms;  
}
```

4.3.2 Verificación de Expiración

```
int scheduler_rr_is_quantum_expired(scheduler_t *scheduler) {  
    struct timespec current_time;  
    clock_gettime(CLOCK_MONOTONIC, &current_time);  
  
    long elapsed_ms = time_diff_ms(&scheduler-  
>quantum_control.quantum_start_time,  
                                   &current_time);  
  
    return elapsed_ms >= scheduler->config.quantum_ms;  
}
```

4.3.3 Manejo de Preemption

```
void scheduler_rr_handle_quantum_expiration(scheduler_t *scheduler) {
    // Obtener producto actual
    product_t *current = scheduler->current_product;

    if (current && current->remaining_time > 0) {
        // Producto no terminó en su quantum
        // Registrar preemption
        scheduler->stats.preemptions++;
        metrics_scheduler_preemption(current->id);

        // Reencolar al final
        queue_push(scheduler->ready_queue, current);

        SCHEDULER_INFO("Producto %d preempted, reinsertado en cola",
            current->id);
    }

    scheduler->current_product = NULL;
}
```

4.3.4 Procesamiento Round Robin

```
void scheduler_rr_process(scheduler_t *scheduler) {
    while (scheduler->thread_running) {
        // Seleccionar siguiente producto
        product_t *product = queue_pop(scheduler->ready_queue);

        if (product) {
            // Iniciar quantum
            scheduler_rr_start_quantum(scheduler, product);
            scheduler->current_product = product;

            // Despachar
            scheduler_dispatch_product(scheduler, product);

            // Simular procesamiento con verificación de quantum
            while (!scheduler_rr_is_quantum_expired(scheduler) &&
                product->remaining_time > 0) {
                usleep(100000); // 100ms
                product->remaining_time -= 100;
            }

            // Verificar si expiró quantum
            if (scheduler_rr_is_quantum_expired(scheduler) &&
                product->remaining_time > 0) {
                scheduler_rr_handle_quantum_expiration(scheduler);
            }

            scheduler->stats.total_products_scheduled++;
        }
    }
}
```



```
}  
}
```

4.4 Strategy Pattern para Algoritmos

Archivos a crear:

- `patterns/strategy.c`

Tareas específicas:

4.4.1 Estructura de Estrategia

```
struct scheduling_strategy {  
    product_t* (*select_next)(scheduler_t *scheduler);  
    void (*process)(scheduler_t *scheduler);  
    void (*handle_quantum)(scheduler_t *scheduler);  
    char name[50];  
};
```

4.4.2 Estrategia FCFS

```
scheduling_strategy_t *create_fcfs_strategy(void) {  
    scheduling_strategy_t *strategy =  
    malloc(sizeof(scheduling_strategy_t));  
    strategy->select_next = scheduler_fcfs_select_next;  
    strategy->process = scheduler_fcfs_process;  
    strategy->handle_quantum = NULL; // FCFS no usa quantum  
    strcpy(strategy->name, "FCFS");  
    return strategy;  
}
```

4.4.3 Estrategia Round Robin

```
scheduling_strategy_t *create_round_robin_strategy(int quantum_ms) {  
    scheduling_strategy_t *strategy =  
    malloc(sizeof(scheduling_strategy_t));  
    strategy->select_next = scheduler_rr_select_next;  
    strategy->process = scheduler_rr_process;  
    strategy->handle_quantum = scheduler_rr_handle_quantum_expiration;  
    strcpy(strategy->name, "Round Robin");  
    return strategy;  
}
```

4.4.4 Cambio Dinámico de Estrategia

```
void scheduler_set_strategy(scheduler_t *scheduler, scheduling_strategy_t
*strategy) {
    pthread_mutex_lock(&scheduler->mutex);

    if (scheduler->strategy) {
        free(scheduler->strategy);
    }

    scheduler->strategy = strategy;
    SCHEDULER_INFO("Estrategia cambiada a: %s", strategy->name);

    pthread_mutex_unlock(&scheduler->mutex);
}
```

4.5 Integración del Scheduler en Main

Archivo a modificar:

- `main.c`

Tareas específicas:

4.5.1 Crear Scheduler

```
// Crear scheduler con algoritmo FCFS
scheduler_t *scheduler = create_scheduler(SCHED_FCFS, 0);

// 0 con Round Robin (quantum 2 segundos)
scheduler_t *scheduler = create_scheduler(SCHED_ROUND_ROBIN, 2000);

// Configurar primera estación
scheduler_set_first_station(scheduler, cutting);
```

4.5.2 Agregar Productos al Scheduler

```
// Crear productos
for (int i = 0; i < 10; i++) {
    product_t *p = factory_create_product(factory);
    scheduler_add_product(scheduler, p);
}
```

4.5.3 Iniciar Procesamiento

```
// Iniciar scheduler
scheduler_start_thread(scheduler);
```

```
// Esperar completitud
scheduler_wait_completion(scheduler);

// Detener
scheduler_stop_thread(scheduler);
```

4.5.4 Comparar Algoritmos

```
// Ejecutar con FCFS
printf("\n=== Prueba con FCFS ===\n");
run_simulation_with_algorithm(SCHED_FCFS, 0);

// Ejecutar con Round Robin
printf("\n=== Prueba con Round Robin ===\n");
run_simulation_with_algorithm(SCHED_ROUND_ROBIN, 2000);

// Comparar métricas
compare_scheduling_algorithms();
```

4.6 Métricas de Scheduling

Métricas a calcular:

4.6.1 Tiempo de Espera

Wait Time = Processing Start Time - Arrival Time
Average Wait Time = Sum(Wait Times) / Number of Products

4.6.2 Tiempo de Turnaround

Turnaround Time = Completion Time - Arrival Time
Average Turnaround = Sum(Turnaround Times) / Number of Products

4.6.3 Tiempo de Respuesta

Response Time = First Execution Time - Arrival Time
Average Response = Sum(Response Times) / Number of Products

4.6.4 Context Switches

Total Context Switches = Número de cambios entre productos

4.6.5 Throughput

$$\text{Throughput} = \text{Products Completed} / \text{Total Time}$$

Fase 5: Añadir IPC y Sincronización Robusta

Objetivo

Fortalecer el sistema de comunicación entre procesos y garantizar sincronización robusta libre de deadlocks y race conditions.

5.1 Utilidades de Tiempo

Archivos a crear:

- `utils/time_utils.h` (ya definido)
- `utils/time_utils.c`

Tareas específicas:

5.1.1 Funciones Básicas de Tiempo

```
void get_current_time(struct timespec *ts);
long timespec_diff_ms(const struct timespec *start, const struct timespec *end);
void timespec_add_ms(struct timespec *ts, long milliseconds);
void sleep_ms(int milliseconds);
```

5.1.2 Quantum Timer

```
typedef struct {
    struct timespec start_time;
    int quantum_ms;
    int expired;
} quantum_timer_t;

quantum_timer_t *create_quantum_timer(int quantum_ms);
void start_quantum(quantum_timer_t *timer);
int is_quantum_expired(quantum_timer_t *timer);
int get_remaining_quantum_ms(quantum_timer_t *timer);
```

5.1.3 Intervalos de Tiempo

```
typedef struct {
    struct timespec start;
    struct timespec end;
    int running;
} time_interval_t;

time_interval_t *create_time_interval(void);
void start_time_interval(time_interval_t *interval);
void end_time_interval(time_interval_t *interval);
long get_interval_duration_ms(const time_interval_t *interval);
```

5.2 Utilidades de Sincronización

Archivos a crear:

- `utils/sync_utils.h` (ya definido)
- `utils/sync_utils.c`

Tareas específicas:

5.2.1 Semáforos con Estadísticas

```
typedef struct {
    sem_t semaphore;
    int initial_value;
    int wait_count;
    int post_count;
    pthread_mutex_t stats_mutex;
} counted_semaphore_t;

counted_semaphore_t *create_counted_semaphore(int initial_value);
int counted_sem_wait(counted_semaphore_t *csem);
void counted_sem_post(counted_semaphore_t *csem);
void print_semaphore_stats(const counted_semaphore_t *csem);
```

5.2.2 Mutex con Timeout

```
typedef struct {
    pthread_mutex_t mutex;
    pthread_cond_t condition;
    int is_locked;
    pthread_t owner_thread;
    struct timespec lock_time;
} timed_mutex_t;

timed_mutex_t *create_timed_mutex(void);
int timed_mutex_lock(timed_mutex_t *tmutex, int timeout_ms);
```

```
int timed_mutex_trylock(timed_mutex_t *tmutex);  
void timed_mutex_unlock(timed_mutex_t *tmutex);
```

5.2.3 Barreras Reutilizables

```
typedef struct {  
    pthread_barrier_t barrier;  
    int thread_count;  
    int current_cycle;  
    int total_synchronizations;  
} reusable_barrier_t;  
  
reusable_barrier_t *create_reusable_barrier(int thread_count);  
int reusable_barrier_wait(reusable_barrier_t *barrier);  
void reset_barrier_cycle(reusable_barrier_t *barrier);
```

5.2.4 Contadores Atómicos

```
typedef struct {  
    pthread_mutex_t mutex;  
    int value;  
    int max_waiters;  
    int current_waiters;  
} atomic_counter_t;  
  
atomic_counter_t *create_atomic_counter(int initial_value);  
int atomic_counter_increment(atomic_counter_t *counter);  
int atomic_counter_decrement(atomic_counter_t *counter);  
int atomic_counter_get(atomic_counter_t *counter);
```

5.3 Mejoras en Colas (IPC Robusto)

Archivo a mejorar:

- `core/queue.c`

Tareas específicas:

5.3.1 Cola con Tamaño Limitado

```
typedef struct {  
    // ... campos existentes ...  
    int max_size;  
    int bounded;  
    sem_t slots_available; // Para cola acotada  
} bounded_queue_t;
```

```
int queue_init_bounded(queue_t *q, int max_size);  
int queue_push_bounded(queue_t *q, product_t *p, int timeout_ms);
```

5.3.2 Operaciones con Timeout

```
product_t* queue_pop_timed(queue_t *q, int timeout_ms);  
int queue_push_timed(queue_t *q, product_t *p, int timeout_ms);
```

5.3.3 Inspección Sin Bloqueo

```
product_t* queue_peek(queue_t *q); // Ver sin remover  
int queue_size(queue_t *q);  
int queue_is_empty(queue_t *q);  
int queue_is_full(queue_t *q);
```

5.3.4 Operaciones Batch

```
int queue_push_batch(queue_t *q, product_t **products, int count);  
int queue_pop_batch(queue_t *q, product_t **buffer, int max_count);
```

5.4 Detección y Prevención de Deadlock

Archivo a crear:

- `utils/deadlock_detector.h/c`

Tareas específicas:

5.4.1 Registro de Recursos

```
void register_resource(const char *resource_name, void *resource_ptr);  
void acquire_resource(pthread_t thread_id, const char *resource_name);  
void release_resource(pthread_t thread_id, const char *resource_name);
```

5.4.2 Detección de Ciclos

```
int detect_deadlock_cycle(void);  
void print_deadlock_report(void);
```

5.4.3 Timeout Automático

```
int wait_with_deadlock_detection(pthread_mutex_t *mutex, int timeout_ms);
```

5.5 Manejo de Señales para Preemption

Archivo a crear:

- `core/signal_handler.h/c`

Tareas específicas:

5.5.1 Configurar Señales

```
void setup_quantum_signal_handler(void);  
void setup_shutdown_signal_handler(void);
```

5.5.2 Handler de SIGALRM para Quantum

```
void quantum_signal_handler(int signum) {  
    // Notificar al scheduler que el quantum expiró  
    // Usar señalización thread-safe  
}
```

5.5.3 Temporizador de Quantum

```
void start_quantum_timer(int quantum_ms);  
void stop_quantum_timer(void);
```

5.6 Buffer Circular Thread-Safe

Archivo a crear:

- `core/circular_buffer.h/c`

Tareas específicas:

5.6.1 Estructura del Buffer

```
typedef struct {  
    product_t **buffer;  
    int head;  
    int tail;  
    int size;  
    int count;
```



```
pthread_mutex_t mutex;  
pthread_cond_t not_empty;  
pthread_cond_t not_full;  
} circular_buffer_t;
```

5.6.2 Operaciones

```
circular_buffer_t *create_circular_buffer(int size);  
int circular_buffer_put(circular_buffer_t *cb, product_t *product);  
product_t *circular_buffer_get(circular_buffer_t *cb);  
int circular_buffer_is_empty(circular_buffer_t *cb);  
int circular_buffer_is_full(circular_buffer_t *cb);
```

5.7 Pruebas de Robustez

Verificaciones necesarias:

5.7.1 Stress Testing

- Ejecutar con 100+ productos
- Ejecutar con quantum muy pequeño (100ms)
- Ejecutar con tiempos de procesamiento variables
- Verificar que no haya memory leaks

5.7.2 Deadlock Testing

- Provocar condiciones de deadlock intencionalmente
- Verificar que el detector funcione
- Verificar que timeouts eviten bloqueos permanentes

5.7.3 Race Condition Testing

- Usar herramientas: `valgrind --tool=helgrind`
- Usar Thread Sanitizer: compilar con `-fsanitize=thread`
- Verificar consistency de métricas

5.7.4 Performance Testing

- Medir overhead de sincronización
- Comparar throughput FCFS vs Round Robin
- Medir context switch overhead
- Analizar utilización de CPU

Resumen de Archivos Pendientes

Fase 3 - Concurrency

- ☐ `core/station.c` - Completar implementación

- ☐ `patterns/chain_handler.c` - Implementar completamente
- ☐ `main.c` - Modificar para usar hilos
- ☐ `Makefile` - Agregar `-DUSE_THREADING`

Fase 4 - Scheduling

- ☐ `core/scheduler.c` - Implementación completa
- ☐ `patterns/strategy.c` - Implementar FCFS y Round Robin
- ☐ `main.c` - Integrar scheduler

Fase 5 - IPC y Sincronización

- ☐ `utils/time_utils.c` - Implementar utilidades
- ☐ `utils/sync_utils.c` - Implementar wrappers
- ☐ `core/queue.c` - Mejorar con bounded queues
- ☐ `utils/deadlock_detector.c` - Sistema de detección
- ☐ `core/signal_handler.c` - Manejo de señales
- ☐ `core/circular_buffer.c` - Buffer circular

Orden de Implementación Recomendado

1. **Fase 3.1-3.2:** Estaciones como hilos + Activar threading
2. **Fase 3.3:** Chain of Responsibility
3. **Fase 3.4:** Integrar en main
4. **Fase 3.5:** Pruebas básicas de concurrencia
5. **Fase 4.1-4.2:** Scheduler base + FCFS
6. **Fase 4.3-4.4:** Round Robin + Strategy Pattern
7. **Fase 4.5:** Integración scheduler en main
8. **Fase 4.6:** Métricas de scheduling
9. **Fase 5.1:** Utilidades de tiempo
10. **Fase 5.2:** Utilidades de sincronización
11. **Fase 5.3:** Mejoras en colas
12. **Fase 5.4-5.6:** Detección deadlock y buffers
13. **Fase 5.7:** Pruebas exhaustivas de robustez